

Brute Force Algorithm

اعداد الطلاب /
يحيى صالح عبد الله اليميني
الياس عمر عبد الله محمد السوداني
عبد اللطيف نريد صالح قيراط
احمد علي صالح النقيب
عبد الوهاب حسين صالح البيل
عقيل حسين علي مشكل



[illegible]

تعريف خوارزمية القوة الغاشمة :

هي طريقة لحل المشكلات تعتمد على تجربة جميع الحلول الممكنة حتى يتم العثور على الحل الصحيح. تعتبر هذه الطريقة فعالة لكنها قد تكون بطيئة جدًا في المشكلات التي تحتوي على عدد كبير من الحلول الممكنة.

اهية خوارزمية القوة الغاشمة (Brute Force Algorithm)



١. أساس للتعليم والبرمجة

- (١) **الحلقات (Loops):** التي تُستخدم لتجربة كل الاحتمالات.
- (٢) **التفكير المنطقي:** تحليل المشكلة وتحديد جميع الحلول الممكنة بشكل منهجي.
- (٣) **التعقيد الزمني:** فهم كيف يمكن لزيادة حجم المدخلات أن تؤثر بشكل كبير على أداء الخوارزمية، مما يمهد الطريق لفهم خوارزميات أكثر كفاءة.



٢. ضمان إيجاد الحل

- (١) **المشكلات الصغيرة:** التي يكون فيها عدد الاحتمالات محدودًا، مثل فك كلمات مرور قصيرة أو حل ألغاز بسيطة.
- (٢) **كأساس للمقارنة:** تُستخدم نتائج خوارزمية القوة الغاشمة كخط أساس (baseline) لمقارنة أداء الخوارزميات الأكثر تعقيدًا. إذا كانت خوارزمية جديدة أسرع من القوة الغاشمة، فإنها تُعتبر فعالة.

أهمية خوارزمية القوة الغاشمة (Brute Force Algorithm)



٣. بوابات لمفاهيم متقدمة

تُعد هذه الخوارزمية المدخل لفهم خوارزميات أكثر تطوراً تُعرف باسم خوارزميات "الخلفية" (Backtracking و "التحسين" Optimization). هذه الخوارزميات تقوم بتحسين منهج القوة الغاشمة من خلال:

١. إزالة المسارات غير المجدية: لا تواصل البحث في مسارات معينة إذا اتضح أنها لن تؤدي إلى حل.

٢. الاستفادة من هياكل البيانات: استخدام هياكل بيانات معقدة لتقليل عدد العمليات المطلوبة.

مدخل الى خوارزمية القوة الغاشمة (Brute Force Algorithm)

تخيل ان معك مفتاح لقفل مكون من ٤ أرقام، لكنك لا تعرف الرقم الصحيح.  ماذا ستفعل؟ ستجرب ... 0002 → 0001 → 0000 حتى تصل للرقم الصحيح. هذه بالضبط فكرة خوارزمية القوة الغاشمة:

نحرب كل الاحتمالات → حتى نجد الحل الصحيح. 

السري في قوتها : البساطة والوضوح.

العيب: قد تكون بطيئة جدًا إذا كانت الاحتمالات كثيرة



خوارزمية القوة الغاشمة (Brute Force Algorithm)

أمثلة من الواقع :



١. البحث في كل الملفات على الحاسوب حتى تجد الملف المطلوب . 

٢. تجربة جميع المفاتيح لفتح باب . 

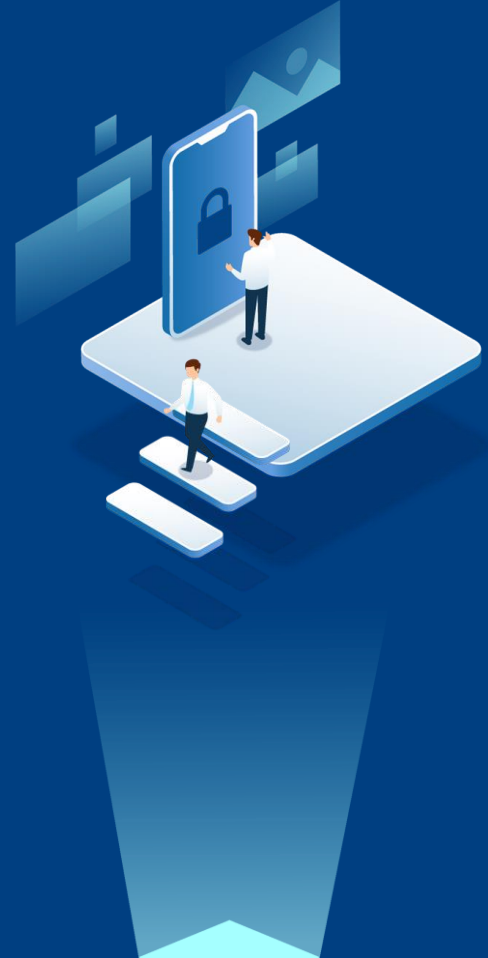
٣. حل لغز بتركيب القطع واحدة تلو الأخرى . 



يعد أسلوب القوة الغاشمة (Brute Force Algorithm) طريقة مباشرة وبديهية لحل المشكلات عن طريق فحص كل الحلول الممكنة بشكل منهجي حتى يتم العثور على الحل الصحيح. على الرغم من أنها قد تكون غير فعالة للمشاكل الكبيرة، إلا أنها تتميز بالبساطة وتضمن إيجاد الحل إذا كان موجودًا.

طرق التنفيذ الأساسية
يمكن تلخيص طرق تنفيذ خوارزمية القوة الغاشمة في أربع خطوات رئيسية

- * أولاً : يتم إنشاء جميع الحلول الممكنة للمشكلة.
- * ثانياً : الفحص : (Test) يتم اختبار كل حل تم إنشاؤه للتحقق مما إذا كان يفي بشروط المشكلة.
- * ثالثاً : التحقق : (Valid) يتم التحقق مما إذا كان الحل الحالي هو الحل المطلوب.
- * رابعاً : الإخراج : (Output) يتم إخراج الحل الصحيح بمجرد العثور عليه



خوارزمية القوة الغاشمة (Brute Force Algorithm)

تطبيقات شائعة

تُستخدم خوارزميات القوة الغاشمة في العديد من المجالات، منها:

* فك شفرات كلمات المرور: *
تجربة كل التوليفات الممكنة من الأحرف والأرقام والرموز.

* ** مشكلة البائع المتجول (Traveling Salesperson Problem): **
حساب طول كل المسارات الممكنة بين المدن لاكتشاف أقصر مسار.

* ** البحث عن الأنماط: **
البحث عن سلسلة نصية معينة داخل نص أكبر من خلال مقارنة كل موقع ممكن.

خوارزمية القوة الغاشمة (Brute Force Algorithm)

شرح احدى التطبيقات :

مشكلة البائع المتجول
(Traveling Salesman Problem)

هذا المثال أبسط نسخة من المشكلة، ويستخدم أسلوب القوة الغاشمة لاستكشاف جميع التباديل الممكنة للمدن لإيجاد أقصر مسار. هذا الحل يكون فعالاً فقط لعدد قليل جداً من المدن بسبب التعقيد العالي.



خوارزمية القوة الغاشمة (Brute Force Algorithm)

```
const int numCities = 4;
int distanceMatrix[numCities][numCities] = {
    { 0, 10, 15, 20 }, { 10, 0, 35, 25 },
    { 15, 35, 0, 30 }, { 20, 25, 30, 0 }
};
```

الكود البرمجي لهذه المشكلة

```
int calculatePathDistance(const vector<int>& path)
{
    int distance = 0;
    for (size_t i = 0; i < path.size() - 1; ++i)
    {
        distance += distanceMatrix[path[i]][path[i + 1]];
    }
    return distance;}

```

خوارزمية القوة الغاشمة (Brute Force Algorithm)

```
int main() {  
    vector<int> citiesToPermute = {1, 2, 3};  
  
    int minDistance = numeric_limits<int>::max();  
    vector<int> bestPath;  
  
    do {  
        vector<int> currentPath = {0};  
        currentPath.insert(currentPath.end(),  
            citiesToPermute.begin(), citiesToPermute.end());  
        currentPath.push_back(0);  
  
        int currentDistance = calculatePathDistance(currentPath);  
    } while (true);  
}
```

خوارزمية القوة الغاشمة (Brute Force Algorithm)

```
if (currentDistance < minDistance) {
    minDistance = currentDistance;
    bestPath = currentPath;
}

} while (next_permutation(citiesToPermute.begin(),
citiesToPermute.end()));

cout << "thii is the Shortest wauy : \n";
for (int city : bestPath) {
    cout << city << " ";
}
cout << "\n >> " المسافة الكلية: minDistance << endl;

return 0;}
```

خوارزمية القوة الغاشمة (Brute Force Algorithm)

مزايا الخوارزمية :

١. البساطة وسهولة التطبيق

تُعد خوارزمية القوة الغاشمة من أبسط الخوارزميات، حيث لا تتطلب تفكيرًا معقدًا أو استخدام هياكل بيانات متقدمة. يسهل فهمها وتطبيقها وبرمجتها، مما يجعلها خيارًا ممتازًا للمبتدئين في تعلم الخوارزميات.

٢. ضمان العثور على الحل الأمثل

الميزة الأهم لهذه الخوارزمية هي أنها تضمن العثور على الحل الأمثل (Optimal Solution) إذا كان موجودًا. بما أنها تقوم بفحص كل الاحتمالات الممكنة، فإنها لا تُغفل أي حل. تُستخدم هذه الخاصية كمرجع للمقارنة مع الخوارزميات الأخرى الأكثر كفاءة. إذا كانت خوارزمية جديدة قادرة على إيجاد حل في وقت أسرع من القوة الغاشمة، فإنها تُعتبر خوارزمية محسّنة.

خوارزمية القوة الغاشمة (Brute Force Algorithm)

مزايا الخوارزمية :

٣. عدم الحاجة لمعرفة مسبقة:

لا تتطلب هذه الخوارزمية أي معلومات مسبقة عن هيكل المشكلة أو طبيعة الحل. إنها تعمل بشكل جيد في المشكلات التي ليس لها بنية واضحة أو التي يصعب فيها تطبيق طرق حل ذكية. على سبيل المثال، في حالات فك كلمات المرور، قد لا تكون هناك طريقة أفضل للعثور على الحل من تجربة كل التوليفات.

٤. مفيدة للمشكلات الصغيرة :

على الرغم من أنها غير فعالة للمشكلات الكبيرة، إلا أن القوة الغاشمة تُعد مثالية للمشكلات التي يكون فيها عدد الاحتمالات صغيرًا ومحدودًا. في هذه الحالات، يكون وقت تنفيذها مقبولًا ولا يستدعي استخدام خوارزميات أكثر تعقيدًا.

خوارزمية القوة الغاشمة (Brute Force Algorithm)

معيوب الخوارزمية :

١ (الكفاءة المنخفضة (Inefficiency) 🐢) :

هذا هو أكبر عيب في خوارزمية القوة الغاشمة. بما أنها تجرب كل حل ممكن، فإنها تصبح بطيئة جدًا وغير عملية مع ازدياد حجم المشكلة. يُعرف هذا النمو السريع باسم التعقيد الزمني الأسّي (Exponential Time Complexity).

مثال:

في مشكلة البائع المتجول، إذا كان لديك ١٠ مدن، فإن عدد المسارات الممكنة هو أكثر من ٣,٦ مليون. ولكن إذا زاد العدد إلى ٢٠ مدينة، فإن عدد المسارات يصبح ضخماً جدًا لدرجة أن أقوى أجهزة الكمبيوتر ستحتاج إلى آلاف السنين لحلها.

خوارزمية القوة الغاشمة (Brute Force Algorithm)

معيوب الخوارزمية :

٢. استهلاك الموارد ( Resource Consumption) :

بسبب طبيعتها التي تتطلب فحص كل احتمال، تستهلك هذه الخوارزمية كمية كبيرة من موارد الحاسوب، مثل:

١. وقت المعالجة: (CPU Time) يتم استهلاك الكثير من وقت المعالج في الحسابات المتكررة.

٢. الذاكرة (Memory) في بعض المشكلات، قد تحتاج الخوارزمية إلى تخزين عدد كبير من

الاحتمالات أو المسارات، مما يستهلك الذاكرة بشكل كبير.

خوارزمية القوة الغاشمة (Brute Force Algorithm)

معيوب الخوارزمية :

٣. عدم قابلية التوسع ( Lack of Scalability) :

لا يمكن تطبيق خوارزمية القوة الغاشمة على المشكلات الكبيرة. فزيادة بسيطة في حجم المشكلة (مثل إضافة مدينة واحدة إلى مشكلة البائع المتجول) تؤدي إلى زيادة هائلة في وقت الحل، مما يجعلها غير مجدية عملياً. هذا يعني أنها ليست قابلة للتوسع للاستخدام في تطبيقات العالم الحقيقي التي تتعامل مع بيانات ضخمة.

٤. غير ضرورية في بعض الأحيان 😞 (Often Unnecessary) :

في كثير من الحالات، توجد خوارزميات أكثر ذكاءً وكفاءة لحل نفس المشكلة. على سبيل المثال، بدلاً من تجربة كل التوليفات لحل مشكلة الفرز (Sorting)، توجد خوارزميات أسرع بكثير مثل الفرز السريع (Quicksort) أو فرز الدمج (Mergesort) .

خوارزمية القوة الغاشمة (Brute Force Algorithm)

طرق تحسين الخوارزمية :

هناك عدة طرق فعّالة لتحسين خوارزمية القوة الغاشمة، وأفضل طريقة تعتمد على طبيعة المشكلة نفسها. التحسين الأساسي لا يعتمد على جعل خوارزمية القوة الغاشمة أسرع، بل على التحول إلى خوارزمية أكثر ذكاءً تتجنب تجربة كل الاحتمالات.

١. التقليم والتراجع (Pruning & Backtracking) ✂ :

هذه هي أفضل وأشهر طريقة لتحسين أداء خوارزميات البحث الشامل (مثل القوة الغاشمة). الفكرة بسيطة: توقف عن البحث في المسارات التي تعلم أنها لن تؤدي إلى حل أمثل.

خوارزمية القوة الغاشمة (Brute Force Algorithm)

طرق تحسين الخوارزمية :

٢. البرمجة الديناميكية (Dynamic Programming) 🧠 :

هذه الطريقة فعّالة عندما يكون لديك مشكلات فرعية متكررة ومتداخلة. بدلاً من إعادة حل نفس المشكلة الفرعية مرارًا وتكرارًا، تقوم بحل كل مشكلة مرة واحدة وتخزين النتيجة.



٣. استخدام هياكل البيانات المناسبة

اختيار هيكل البيانات المناسب يمكن أن يقلل بشكل كبير من التعقيد الزمني.

١. إذا كنت تبحث عن عنصر في قائمة غير مرتبة، فإن القوة الغاشمة (البحث الخطي) هي $O(n)$.

إذا قمت بفرز القائمة، يمكنك استخدام البحث الثنائي ($O(\log n)$).

٢. في مشكلة فحص التكرار، استخدام الخريطة المجزأة (hash map) بدلاً من القائمة العادية

يمكن أن يحسن الأداء من $O(n^2)$ إلى $O(n)$.

خوارزمية القوة الغاشمة (Brute Force Algorithm)

وفي الختام، يمكننا القول إن خوارزمية القوة الغاشمة هي نقطة الانطلاق في عالم حل المشكلات. على الرغم من أنها قد تبدو بطيئة وغير فعالة للمشكلات الكبيرة، إلا أن أهميتها تكمن في بساطتها وضمانها للعثور على الحل الأمثل.

إنها بمثابة الأساس الذي تُبنى عليه الخوارزميات الأكثر تعقيدًا. فهم مبادئها هو الخطوة الأولى نحو استيعاب خوارزميات أكثر ذكاءً وكفاءة مثل التراجع (Backtracking) والبرمجة الديناميكية (Dynamic Programming) .

خوارزمية القوة الغاشمة (Brute Force Algorithm)

اشراف الدكتور /
وليد دحية دكتور مادة البرمجة الكائنية

اعداد الطلاب /
يحيى صالح عبد الله اليمني
الياس عمر عبد الله محمد السوداني
عبد اللطيف نريد صالح قيراط
احمد علي صالح النقيب
عبد الوهاب حسين صالح البيل
عقيل حسين علي مشكل

وشكرا لحضوركم عرضنا التقديمي